Basic (your first FastAPI route) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `health_route_practice.py`: every real service needs one tiny route that answers "are you up?", and it is the smallest FastAPI app possible — so it's the right place to learn the app, the decorator, and uvicorn before any real logic is involved. **If not:** your first FastAPI route would be the Build Task's `/run-task`, with a database and an agent behind it, and any mistake in the basic setup would be buried under all of that.

The solution below runs from inside `practice/` with `uvicorn health_route_practice:app --reload`, and is checked with `curl localhost:8000/health`.

Solution

Story: a bare dict can silently change shape (a typo like `"stauts"`) and no one notices until a client breaks. A `response_model` makes FastAPI check every reply and document it. **If not:** the Build Task's replies would have no written contract, and a wrong field name would only be found by the client.

```
# practice/health_route_practice.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    # why: the reply's shape is written down once, in one place
    status: str

# how: FastAPI checks the return value against HealthResponse
@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    """Liveness check: is the process up and able to answer at all."""
    return HealthResponse(status="ok")
```

Expected output:

```
{"status":"ok"}
```

The JSON reply looks identical to Basic's, but `http://localhost:8000/docs` now shows a documented `HealthResponse` schema for this route, and FastAPI would reject (loudly, with a server-side error) any code path that tried to return something that doesn't match `HealthResponse`.